Universidade Cruzeiro do Sul
Ciência da Computação

Aula 11 - Paradigma Imperativo – Parte 2

Disciplina: Paradigmas de Linguagens de Programação
Professor: Álvaro Colombero

Objetivos



- Definir abstração.
- Aplicar o processo de abstração procedural na resolução de problemas.
- Diferenciar abstração de dados e abstração de processos.

Introdução

O que é abstração?

Por que ela é importante para a programação?



Abstração

- **Abstração** é a habilidade de concentrar nos aspectos essenciais de um contexto qualquer, ignorando características menos importantes.
- Ela possibilita a compreensão dos aspectos relevantes de um problema para que seja possível a construção de uma solução computacional (programa) que o resolva.
- Resumindo, visa simplificar o processo de construção de um programa de computador.

Abstração

- Abstração em linguagens de programação:
  - Abstração de processos (Paradigma Imperativo).
    - Subprogramas (funções).
  - Abstração de dados (Paradigma Orientado a Objetos).
    - Tipos de dados abstratos.

Abstração Procedural

O que é abstração procedural?

É o processo que permite ao programador se preocupar com a interface entre a função e o que ela calcula, ignorando os detalhes de como o cálculo é realizado

Abstração Procedural

- Processo de abstração procedural:
  - Descrever sucintamente a função calculada, junto a sua entrada e a sua saída.
  - Dividir a função descrita em um conjunto de funções mais primitivas, com suas respectivas entradas e saídas.
  - Esse processo é repetido até que todas as funções a serem calculadas e seus dados possam ser executados pelos comandos e tipos de dados primitivos da linguagem.

Abstração Procedural

Exemplo: Faça um programa que leia um conjunto de dez números inteiros, apresente estes dados e, em seguida, ordene-os e apresente-os novamente.

Quais passos devem ser realizados por este programa?

Abstração Procedural

- Abstração 1:
  - Ler os dez valores e armazená-los em um vetor.
  - Imprimir o vetor original.
  - Ordenar o vetor.
  - Imprimir o vetor ordenado.

Abstração Procedural

- Abstração 1:
  - Ler os dez valores e armazená-los em um vetor.
  - Imprimir o vetor original.
  - Ordenar o vetor.
  - Imprimir o vetor ordenado.

```
33  int main() {
34      int vetor[10];
35      lerVetor(vetor);
36      mostrarVetor(vetor);
37      ordenarVetor(vetor);
38      mostrarVetor(vetor);
39      return 0;
40  }
```

Abstração Procedural

- Como os processos de leitura e impressão de um vetor são simples, eles podem ser codificados diretamente.

```
3  void lerVetor(int* vetor){
4      for (int i=0;i<10;i++){
5          scanf("%d",&vetor[i]);
6      }
7  }
```

```
27 void mostrarVetor(int* vetor){
28     for (int i=0;i<10;i++){
29         printf("Elemento da posição %d: %d\n",i, vetor[i]);
30     }
31 }
```

Abstração Procedural

- Abstração 2: ordenarVetor.
  - Percorrer o vetor completo e encontrar o menor valor.
  - Trocar o menor valor e o primeiro elemento de posição.
  - Repetir este processo iniciando na posição seguinte (2, 3, …) até atingir o final do vetor.

```
25  void ordenarVetor(int* vetor){
26      int posMenor;
27      for (int i=0;i<10;i++){
28          posMenor = encontrarMenor(vetor, i);
29          trocarElementos(&vetor[i],&vetor[posMenor]);
30      }
31  }
```

Abstração Procedural

- Abstração 3: encontrarMenor
  - Considerar o primeiro elemento como o menor e armazenar a sua posição.
  - Percorrer o vetor e comparar os elementos com o menor.
  - Quando um novo elemento for menor que o atual, atualizar a variável menor com este valor.

```
 9 int encontrarMenor(int* vetor, int inicio) {
10     int posMenor = inicio;
11     for (int j=inicio+1;j<10;j++){
12         if (vetor[j]<vetor[posMenor]){
13             posMenor = j;
14         }
15     }
16     return posMenor;
```

Abstração Procedural

- Abstração 3: trocarElementos
  - Salvar um dos valores em uma variável temporária.
  - Substituir o valor da variável salva pelo novo valor;
  - Substituir o valor copiado pelo salvo na variável temporária.

```
19  void trocarElementos(int *a, int *b){
20      int temp = *a;
21      *a = *b;
22      *b = temp;
23  }
```

Abstração Procedural

- A abstração procedural permite:
  - Organizar melhor o código fonte, melhorando a sua legibilidade de manutenibilidade (estes aspectos foram melhorados significativamente com a OO).
  - Reutilizar funções comuns em diferentes pontos de um mesmo programa, reduzindo o trabalho do programador.
  - Reutilizar funções comuns em diferentes programas, por meio de bibliotecas.
  - Separar os módulos de um programa, conforme a sua função (financeiro, clientes, estoque, etc).
  - Criar aplicações separadas que interajam para a resolução do problema (frontend e backend, por exemplo).

Abstração Procedural

- A abstração procedural não favorece o baixo acoplamento :
  - Acoplamento está relacionado ao grau de dependência entre diferentes módulos de um sistema.
  - Variáveis globais geram um alto acoplamento, por isto devem ser evitadas.
- A abstração procedural não favorece a baixa coesão:
  - Coesão está relacionada ao princípio da responsabilidade única e determina que uma função (ou classe) deve ter apenas uma única responsabilidade.
- Tais aspectos dificultam a criação de códigos com boa legibilidade e manutenibilidade.

Abstração de Dados

- “Um tipos abstrato de dados é um invólucro que inclui apenas a representação de dados de um tipo de dados específico e os subprogramas que fornecem as operações para este tipo” (SEBESTA, 2010).
- “Encapsulamento é o mecanismo que permite que constantes logicamente relacionadas, tipos, variáveis, métodos, entre outros, sejam agrupadas em uma nova entidade” (Tucker e Noonan, 2008).
- Por meio de mecanismos de controle de acesso, detalhes desnecessários são ocultados das demais entidades que usam este invólucro.

Abstração de Dados

- Tipos de dados abstratos são tipos que:
  - Permitem a declaração de variáveis deste tipo.
  - Ocultam a representação interna dos tipos.
  - Possuem um conjunto de operações para manipular as variáveis criadas.
  - As declarações do tipo e das operações estão definidas na mesma unidade sintática, embora sua implementação pode estar em outra unidade.
  - A representação interna das variáveis é ocultada das unidades do programa que as utilizam, sendo as suas operações as únicas formas para acessá-las.

Exercício

- Escreva um programa em linguagem C que realize o processamento de notas de uma turma.
- O programa deve seguir os seguintes passos na função main, dividindo cada responsabilidade em funções/procedimentos específicos:
  - Ler 10 notas inteiras de alunos (de 0 a 100) e armazená-las em um vetor.
  - Exibir as notas digitadas no formato original.
  - Inverter a ordem dos elementos do vetor (sem criar um vetor auxiliar, alterando o próprio vetor original).
  - Exibir o vetor com as notas invertidas.
- Requisitos de Abstração Procedural:
  - A função principal (main) deve apenas orquestrar as chamadas das funções superiores (lerVetor, mostrarVetor, inverterVetor).
  - A função inverterVetor deve delegar a troca dos elementos opostos para uma função auxiliar dedicada chamada trocarElementos.



Atividade 1

- Faça um programa que leia a quantidade total de elementos de um conjunto e a quantidade de elementos que serão agrupados em arranjos e combinações. Em seguida, calcule e apresente a quantidade de arranjos e combinações que podem ser criados, usando as fórmulas abaixo. Utilize o processo de abstração procedural para resolver o problema.
- O programa pode ser desenvolvido em qualquer linguagem, exceto Java, e se a linguagem suportar OO, tais recursos não devem ser utilizados.

$$A_{n,p} = \frac{n!}{(n-p)!}$$

$$C_{n,p} = \frac{n!}{p!(n-p)!}$$